

CCAO-DSL: Practical Examples and Patterns

Companion to: dsl-specification.md **Version:** 1.0.0 **Date:** 2025-10-19

Table of Contents

1. [Quick Start Examples](#) 2. [Common Patterns](#) 3. [Real-World Workflows](#) 4. [Anti-Patterns](#) 5. [Performance Optimization](#) ---

1. Quick Start Examples

1.1 Single Agent Execution

```
// Simplest form: invoke one agent
api-architect
```

What happens: 1. Parse: Recognize `api-architect` as agent reference 2. Type check: Validate agent exists in registry 3. Execute: Run agent with current context 4. Return: Agent's output **Use case:** Quick one-off tasks, simple questions ---

1.2 Agent with Skills

```
// Load agent with specific domain knowledge
api-architect + rest-api-design-patterns + postgresql
```

What happens: 1. Parse: Agent reference + skill combination 2. Type check: Verify skills compatible with agent 3. Load: Import skill modules into agent context 4. Execute: Agent now

has specialized knowledge **Use case**: When you need an agent with specific expertise ---

1.3 Sequential Pipeline

```
// Chain agents: output of one feeds next
deep-researcher -> api-architect -> practical-programmer
```

What happens: 1. `deep-researcher` analyzes requirements → outputs research 2.

Research → `api-architect` designs API → outputs spec 3. Spec → `practical-`

`programmer` implements → outputs code **Execution order**: Sequential (total time = sum of

agent times) **Use case**: Multi-stage workflows where each step depends on previous ---

1.4 Parallel Execution

```
// Run agents concurrently, merge results
frontend-specialist || backend-specialist || devops-engineer
```

What happens: 1. Fork: Three agents start simultaneously 2. Execute: Each runs independently 3. Join: Wait for all to complete 4. Merge: Combine outputs into unified result

Execution time: $\max(\text{agent times})$, not sum **Use case**: Independent tasks that can run concurrently ---

1.5 Mixed Composition

```
// Combine sequence and parallel
research -> (design || implement) -> integrate
```

Execution flow:

```
research
|
├── design ───┐
|              └──┬── integrate
└── implement ─┘
```

What happens: 1. `research` runs first (sequential) 2. `design` and `implement` run in parallel 3. Wait for both to complete 4. `integrate` runs last (sequential) **Use case:** Complex workflows with dependencies and parallelism ---

2. Common Patterns

2.1 Fan-Out/Fan-In

Problem: Process multiple items in parallel, then aggregate.

```
workflow batch_processor {  
  // Single input splits to multiple processors  
  input -> (  
    processor1 ||  
    processor2 ||  
    processor3  
  ) -> aggregator  
}
```

Visual:

```
input  
  |  
  ├──> processor1 ─┐  
  ├──> processor2 ─┴──> aggregator  
  └──> processor3 ─┐
```

Real example:

```
// Process multiple files in parallel  
file_reader -> (  
  json_validator ||  
  schema_checker ||  
  security_scanner  
) -> report_generator
```

2.2 Map-Reduce

Problem: Apply operation to collection, then reduce.

```
workflow analyze_codebase {  
  // Map: analyze each file  
  files = glob("**/*.py")  
  analysis = map(files, code_analyzer)  
  
  // Reduce: combine results  
  report = reduce(analysis, merge_reports)  
}
```

Pseudo-implementation:

```
# Map phase (parallel)  
analyses = await asyncio.gather(*[  
    analyze(file) for file in files  
)  
  
# Reduce phase (sequential)  
final_report = functools.reduce(merge_reports, analyses)
```

2.3 Pipeline with Context Sharing

Problem: Pass shared context through pipeline.

```
workflow api_development {  
  // Build context progressively  
  ctx = /ctx7("fastapi")  
  ctx = ctx + /ctx7("postgresql")  
  
  // Use context in pipeline  
  researcher[ctx] ->  
  architect[ctx] ->  
  implementer[ctx]  
}
```

Context accumulation:

```
Step 1: ctx = {fastapi_docs}
Step 2: ctx = {fastapi_docs, postgresql_docs}
Step 3: All agents see full context
```

2.4 Conditional Branching (Future)

Problem: Choose agent based on condition.

```
// Future syntax
workflow adaptive_processing {
  result = analyzer

  next = match result.complexity {
    "simple" -> simple_handler
    "medium" -> standard_handler
    "complex" -> expert_handler + deep_analysis
  }

  next(result)
}
```

2.5 Error Recovery

Problem: Handle failures gracefully.

```
workflow resilient_workflow {  
  primary = api_architect  
    .timeout(300)  
    .retry(max=3, backoff=exponential)  
  
  fallback = cached_architect  
  
  result = try(primary) catch fallback  
}
```

Execution logic:

```
try:  
    result = await execute_with_retry(api_architect, max_retries=3)  
except (TimeoutError, RuntimeError):  
    result = load_cached_result()
```

3. Real-World Workflows

3.1 Full-Stack Application Development

```
workflow fullstack_app {
  name: "Complete Application Development"
  version: "1.0.0"
  requires: [react, fastapi, postgresql]

  // Phase 1: Requirements & Research
  requirements = (
    /ctx7("react") ||
    /ctx7("fastapi") ||
    /ctx7("postgresql")
  )

  // Phase 2: Architecture Design
  architecture = deep-researcher + requirements ->
    api-architect + rest-api-design-patterns + postgresql

  // Phase 3: Parallel Implementation
  implementation = (
    // Frontend
    frontend-specialist + react-development + tailwind-css ||

    // Backend
    practical-programmer + fastapi + sqlalchemy ||

    // Database
    database-specialist + postgresql-database-engineering
  )

  // Phase 4: Integration
  integration = git-genius + ci-cd-pipeline-patterns

  // Phase 5: Testing
  testing = (
    test-engineer + jest-react-testing ||
    test-engineer + pytest
  )
}
```

```
// Phase 6: Deployment
deployment = devops-engineer + kubernetes-orchestration

// Complete flow
requirements ->
  architecture ->
    implementation ->
      integration ->
        testing ->
          deployment
}
```

Estimated execution time:

```
Sequential parts: requirements(2m) + architecture(5m) + integration(3m) + deployment(5m)
Parallel parts: max(implementation)=10m, max(testing)=8m = 18m
Total: ~33 minutes (vs ~60+ minutes if fully sequential)
```

3.2 API Design & Documentation

```
workflow api_documentation {
  name: "OpenAPI Spec Generation"
  version: "1.0.0"

  // Research existing APIs
  research = deep-researcher + rest-api-design-patterns

  // Design endpoints
  design = api-architect +
    rest-api-design-patterns +
    postgresql-database-engineering +
    oauth2-authentication

  // Generate artifacts in parallel
  artifacts = (
    // OpenAPI spec
    openapi_generator ||

    // Database schema
    schema_generator + postgresql ||

    // Security documentation
    security_auditor + oauth2-authentication ||

    // Code examples
    example_generator + fastapi
  )

  // Validate everything
  validation = (
    openapi_validator ||
    schema_validator ||
    security_checker
  )

  // Complete pipeline
  research -> design -> artifacts -> validation
}
```

Output: - `openapi.yaml` : Complete API specification - `schema.sql` : Database DDL - `security.md` : Security documentation - `examples/` : Code samples ---

3.3 Code Review & Refactoring

```
workflow code_review {
  name: "Comprehensive Code Review"
  version: "1.0.0"

  // Parallel analysis
  analysis = (
    // Code quality
    practical-programmer + pytest-patterns ||

    // Security audit
    security-expert + oauth2-authentication ||

    // Performance check
    performance-specialist + profiling-tools ||

    // Architecture review
    api-architect + microservices-patterns
  )

  // Aggregate findings
  report = report-generator

  // Generate refactoring plan
  refactor_plan = practical-programmer +
    clean-code-principles

  // Pipeline
  analysis -> report -> refactor_plan
}
```

Output structure:

```
review_results:
  code_quality:
    score: 85/100
    issues: [...]
    suggestions: [...]

  security:
    vulnerabilities: []
    recommendations: [...]

  performance:
    bottlenecks: [...]
    optimizations: [...]

  architecture:
    patterns_used: [...]
    improvements: [...]

  refactor_plan:
    priority_high: [...]
    priority_medium: [...]
    priority_low: [...]
```

3.4 Database Migration Workflow

```
workflow database_migration {
  name: "Safe Database Migration"
  version: "1.0.0"
  requires: [postgresql, alembic]

  // Backup current state
  backup = database-specialist + postgresql ->
    backup_manager

  // Design migration
  migration_design = (
    // Schema changes
    schema_designer + postgresql-database-engineering ||

    // Data transformations
    data_engineer + pandas ||

    // Index optimization
    performance_specialist + database-optimization
  )

  // Generate migration scripts
  scripts = migration_generator + alembic

  // Validation (dry-run)
  validation = (
    // Test on staging
    staging_tester ||

    // Performance impact
    performance_analyzer ||

    // Rollback plan
    rollback_planner
  )

  // Apply migration
  apply = database-specialist + alembic ->
    migration_applier
}
```

```
// Complete flow with safety checks
backup ->
  migration_design ->
  scripts ->
  validation ->
  apply
}
```

3.5 Machine Learning Pipeline

```
workflow ml_pipeline {
  name: "End-to-End ML Pipeline"
  version: "1.0.0"
  requires: [pandas, scikit-learn, mlflow]

  // Data preparation
  data_prep = (
    data_engineer + pandas ||
    feature_engineer + scikit-learn
  )

  // Model training (try multiple models)
  training = (
    model_trainer + linear_models ||
    model_trainer + tree_models ||
    model_trainer + neural_networks
  )

  // Model evaluation
  evaluation = ml_evaluator + mlops-workflows

  // Best model selection
  selection = model_selector

  // Deployment
  deployment = (
    // Containerize
    devops_engineer + docker-compose-orchestration ||

    // Setup monitoring
    monitoring_specialist + observability-monitoring ||

    // API wrapper
    api_architect + fastapi
  )

  // Complete flow
  data_prep ->
  training ->
  evaluation ->
```

```
selection ->
deployment
}
```

4. Anti-Patterns

4.1 Unnecessary Sequencing

Bad:

```
// Forces sequential execution when parallel is possible
task1 -> task2 -> task3 -> task4
```

Good:

```
// Parallel when no dependencies
task1 || task2 || task3 || task4
```

Impact: Bad version takes 4x longer unnecessarily. ---

4.2 Skill Overloading

Bad:

```
// Loading too many unrelated skills
agent + skill1 + skill2 + skill3 + skill4 + skill5
```

Problems: - Slow initialization - Context confusion - Memory overhead **Good:**

```
// Only load necessary skills
agent + relevant_skill1 + relevant_skill2
```

4.3 Missing Error Handling

Bad:

```
// No error handling - fails completely on any error
critical_task1 -> critical_task2 -> critical_task3
```

Good:

```
// Graceful degradation
workflow resilient {
  result = try(critical_task1)
    .timeout(300)
    .retry(3)
    .fallback(cached_result)

  result -> critical_task2 -> critical_task3
}
```

4.4 Circular Dependencies

Bad:

```
// Creates cycle - INVALID!
workflow broken {
  a = b -> c
  b = c -> d
  c = d -> a // ← cycle!
}
```

Error:

```
CyclicDependencyError: Detected cycle in workflow graph
a → b → c → d → a
```

Good:


```
// Break the cycle
workflow fixed {
  a = b -> c
  b = c -> d
  // d completes without depending on a
}
```

4.5 Resource Exhaustion

Bad:

```
// Spawns too many parallel tasks
agent1 || agent2 || agent3 || ... || agent100
```

Error:

```
ResourceError: Exceeded max_parallel limit (5)
```

Good:

```
// Batch parallel execution
workflow batched {
  batch1 = agent1 || agent2 || agent3 || agent4 || agent5
  batch2 = agent6 || agent7 || agent8 || agent9 || agent10

  batch1 -> batch2 // Sequential batches
}
```

5. Performance Optimization

5.1 Parallelization Strategy

Before:

```
// Sequential: 30 seconds
task1 -> task2 -> task3 // 10s + 10s + 10s
```

After:

```
// Parallel: 10 seconds
task1 || task2 || task3 // max(10s, 10s, 10s)
```

Speedup: 3x ---

5.2 Skill Preloading

Before:

```
// Loads skills every time
workflow repeated {
  agent + skill11 + skill12
  agent + skill11 + skill12
  agent + skill11 + skill12
}
```

After:

```
// Load once, reuse
workflow optimized {
  configured_agent = agent + skill11 + skill12

  configured_agent
  configured_agent
  configured_agent
}
```

Speedup: ~5x for skill loading overhead ---

5.3 Result Caching

Before:

```
// Recomputes every time
expensive_research
expensive_research // Same query!
expensive_research
```

After:

```
// Compute once
workflow cached {
  result = expensive_research.cache(ttl=3600)

  task1(result)
  task2(result)
  task3(result)
}
```

Speedup: 1x + 2 cache hits = massive improvement ---

5.4 Lazy Evaluation

Before:

```
// Loads everything upfront
ctx = /ctx7("lib1") + /ctx7("lib2") + /ctx7("lib3")

// But only uses lib1
agent[ctx].process() // Only needs lib1!
```

After:

```
// Load on demand
agent + lazy_load("lib1", "lib2", "lib3")
```

Speedup: Only loads what's actually used ---

5.5 Work Stealing

Problem: Unbalanced parallel tasks

```
// One task much slower than others
fast_task1 || fast_task2 || slow_task // 2s, 2s, 30s
```

Worker timeline:

```
Worker 1: [fast_task1: 2s] [idle: 28s]
Worker 2: [fast_task2: 2s] [idle: 28s]
Worker 3: [slow_task: 30s]
```

Solution: Work stealing scheduler

```
Worker 1: [fast_task1: 2s] [steal: subtask_a: 14s]
Worker 2: [fast_task2: 2s] [steal: subtask_b: 14s]
Worker 3: [slow_task: split into subtasks]
```

New time: 16s instead of 30s ---

6. Quick Reference

Operator Cheatsheet

Operator	Name	Usage	Example	Combination	Add skills/capabilities
+	Sequence	Pipeline (order matters)	<code>agent + skill1 + skill2</code>	<code> -></code>	
:	Assignment		<code>a b c</code>	<code>:</code>	
=	Definition	Define workflows/agents	<code>result: agent</code>	<code> =</code>	
			<code>workflow = a -> b</code>		

Precedence (highest to lowest)

1. `( )` - Grouping 2. `/command` - Command invocation 3. `[skills]` - Skill loading 4. `+` - Combination 5. `||` - Parallel 6. `->` - Sequence 7. `:`, `=` - Assignment

Common Patterns Quick Reference

```
// Single agent
agent_name

// Agent with skills
agent + skill1 + skill2

// Sequential
a -> b -> c

// Parallel
a || b || c

// Mixed
a -> (b || c) -> d

// Command + agent
/cmd(args) -> agent

// Workflow
workflow name {
  step1 -> step2
}
```

7. Debugging Tips

7.1 Visualize DAG

```
# Generate execution graph
dsl visualize workflow.dsl > graph.dot
dot -Tpng graph.dot > graph.png
```

7.2 Dry Run

```
# Check without executing
dsl validate workflow.dsl --dry-run
```

7.3 Step-by-Step Execution

```
# Execute with breakpoints
dsl debug workflow.dsl --step
```

7.4 Trace Execution

```
# Full execution trace
dsl execute workflow.dsl --trace > trace.log
```

7.5 Type Check

```
# Verify types
dsl typecheck workflow.dsl
```

--- **Document Status:** Examples Complete **Companion:** dsl-specification.md **Version:** 1.0.0